

ЛАБОРАТОРНАЯ РАБОТА № 8

ПОСТРОЕНИЕ СОСТАВНЫХ ЛИСП-ПРОГРАММ. ЦИКЛЫ И РАЗВЕТВЛЕНИЯ

Цель работы: изучение методики построения составных ЛИСП-программ в графической системе *NanoCAD*, операторов цикла и разветвления.

1. ТЕОРЕТИЧЕСКИЙ МАТЕРИАЛ

1.1. Подпрограммы и ветвления

В любом языке программирования есть средства для создания подпрограмм, циклов, ветвлений в программах в зависимости от результатов проверки условий. Такие средства есть и в *Visual LISP*.

Подпрограмма позволяет описать некоторую последовательность действий один раз, а использовать ее в различных местах программы и даже включать в другие программы, что значительно упрощает программирование.

Для этих целей в *Visual LISP* служит уже известная функция *DEFUN*. Описание некоторой функции с помощью *DEFUN* и можно считать подпрограммой, а обращение к описанной ранее функции – вызовом подпрограммы. При обращении на место символических аргументов в описании функции подставляются истинные значения этих переменных. Например, описанная в программе 1 функция *SINCOS(A)* может рассматриваться как подпрограмма. А строка *(SINCOS 0.0)*, встречающаяся в какой-либо программе, является обращением к этой подпрограмме, причем аргумент *A* принимает значение 0.

Ветвления в программе организуются с помощью функции *IF*:

(IF <условие> <функция1> [<функция2>])

Здесь *<условие>* – такое выражение *Visual LISP*, которое среди прочих значений может в какой-то ситуации получить значение *NIL*.

Функция *IF* вычисляет значение условия и, если оно не *NIL*, то выполняет *<функцию1>*, иначе выполняет *<функцию2>*, если она присутствует. Функция *IF* возвращает результат выполнения функции. Таким образом, функция *IF* соответствует управляющей структуре «ЕСЛИ-ТО-ИНАЧЕ».

1.2. Логические функции

Для записи условий чаще всего используются функции, которые назовем логическими. К ним относятся функции

= (равно), < (меньше), > (больше), /= (не равно), <= (меньше или равно), >= (больше или равно).

Эти функции позволяют выполнять сравнение чисел или текстов. Логические функции *AND*, *OR*, *NOT* обеспечивают возможность строить сложные условия из простых.

Все эти функции могут возвращать одно из двух значений: *NIL* («нет», условие не выполняется) или *T* («да», условие выполняется).

Например, функция «*=*» имеет вид: $(= <atom> <atom> \dots)$.

Здесь в качестве атомов могут стоять либо числа (как целые, так и вещественные), либо текстовые данные. Функция «*=*» возвращает *T*, если все атомы равны между собой, иначе – *NIL*.

Функция «*не равно*» определена только для двух атомов.

Функция «*меньше*»: $(< <atom> <atom> \dots)$ возвращает *T*, если каждый предыдущий атом меньше последующего, иначе – *NIL*.

Аналогично определяются остальные функции сравнения. Для текстовых констант понятие «меньше» означает следующее: сравнение производится по первой несовпадающей букве. «Меньшим» считается тот символ, ASCII-код которого меньше. Для букв алфавита коды нарастают в алфавитном порядке (отдельно для прописных, отдельно для строчных букв).

Функции *AND*, *OR*, *NOT* являются функциями алгебры логики.

Функция $(AND <выражение> \dots)$ выполняет операцию логического «И» над списком выражений, т.е. функция возвращает *NIL*, если хотя бы одно выражение имеет значение *NIL*. Иначе возвращается *T*.

Функция $(OR <выражение> \dots)$ выполняет операцию логического «ИЛИ». Эта функция возвращает *T*, если хотя бы одно выражение имеет значение *T*, иначе – *NIL*.

Функция $(NOT <выражение>)$ выполняет логическую функцию «НЕ». Если выражение имеет значение *NIL*, то возвращается *T*, во всех остальных случаях возвращается *NIL*.

Примеры:

Пусть имеются следующие значения:

A-5, *B* – “*TEXT1*”, *C*-*NIL*.

Тогда

$(= A 5)$ возвращает *T*,

$(< B \text{ “TEXT2”})$ – *T*,

$(AND A B C)$ – *NIL*,

$(OR A B C)$ -*T*,

$(NOT C)$ -*T*.

Рассмотрим еще одну логическую функцию *NULL*. Эта функция имеет следующий формат:

$(NULL <аргумент>)$.

Типы аргументов – любые. Тип возвращаемого значения – логическое (*T*, если значение аргумента равно *NIL*, и *NIL* – в противном случае). Фактически идентична функции *NOT*. Традиционно функция *NULL* употребляется для проверки не пустоты списков.

1.3. Методика организации программ с ветвлением и циклами

Приведенная ниже программа 4 иллюстрирует ветвление по условию с использованием функции *IF*.

```
; Программа 4. Вычисление тангенса  
(DEFUN TAN (A / S C) ; Заголовок функции  
; A - аргумент функции - угол, S, C - локальные переменные  
(SETQ S (SIN A))  
(SETQ C (COS A))  
(IF (/ = C 0.0) (/ S C) "бесконечность")  
); Конец функции
```

Здесь, если *C* не равно нулю, вычисляется тангенс (и значение тангенса будет возвращено функцией *TAN*). Если же *C* равно нулю, то тангенс не вычисляется, а функция возвращает строку "бесконечность" – это слово и появится на экране.

Цикл в *Visual LISP* организуется с помощью функции *WHILE*:
(*WHILE* <условие> <выражение>...).

Здесь <условие> – это выражение, которое в некоторых ситуациях может принимать значение *NIL*.

Функция *WHILE* вычисляет значение условия и, если оно не *NIL*, вычисляет выражение, затем снова условие и т.д. Это продолжается, пока условие не станет равно *NIL*. Затем *WHILE* возвращает последнее значение последнего выражения. Рассмотрим пример использования функции *WHILE* для вычисления факториала.

```
Пример:  
; Вычисление факториала  
(SETQ N 10); Задайте N  
(SETQ I 1 FACTORIAL 1)  
(WHILE (< I N)  
(SETQ I (1+ I))  
(SETQ FACTORIAL (* FACTORIAL I))  
); конец WHILE.
```

Рассмотрим работу примера, когда число *N* равно 10 (т.е. вычисляется 10!). В программе используются переменные *I* и *FACTORIAL*. Переменная *I* является счетчиком цикла. Переменная *FACTORIAL* накапливает произведение чисел, формирующее факториал. Перед входом в цикл они получают начальные значения, соответственно 1 и 1 (по определению 1! считается равным 1).

Функция *WHILE* проверяет условие (< *I N*) для текущего значения *I*, и если результат вычисления условия отличен от *NIL*, то выполняет внутренние операции (две операции с участием функции *SETQ*). На первом шаге цикла *I* равно 1. Проверяемое условие (< *I N*) возвращает значение *T* («истина»). Поэтому функция *WHILE* увеличивает *I* на 1 (получается *I* = 2) и умножает переменную *FACTORIAL* на *I*: *FACTORIAL* = 1*2 (не что иное, как 2!).

Далее снова передается управление на вход в цикл (уже при $I = 2$). Проверка условия опять дает результат «истина», поэтому I получает значение 3 ($2+1$), а $FACTORIAL$ — 6 ($2*3$). И так далее, пока I не станет равным N (10). Тогда программа покинет цикл, не выполняя внутренних операций. Результат: при $N=10$ $FACTORIAL = 3628800$.

Приведенная ниже программа 5 иллюстрирует использование цикла, позволяющего компактно записать построение сложного изображения: строится семейство из 20 квадратов, каждый из которых повернут на угол $0.1*\pi$ относительно предыдущего и имеет периметр в 0,9 раз меньше предыдущего. Для построения 20 замкнутых линий используется команда *LINE*.

```
; Программа 5. Построение семейства квадратов:
; Получение исходных данных (описание функции)
(DEFUN ID ( ); Заголовок функции
(SETQ P1 (GETPOINT "\n Нач. точка:"))
(SETQ L (GETDIST P1 "\n Нач. длина:"))
)
; Построение одного квадрата:
(DEFUN QUADR (L A /P2 P3 P4); Заголовок функции
; Здесь P2, P3, P4 объявлены как локальные
; переменные
(SETQ P2 (POLAR P1 A L))
(SETQ P3 (POLAR P2 (+ A (/ PI 2)) L))
(SETQ P4 (POLAR P3 (+ A PI) L))
(COMMAND "LINE" P1 P2 P3 P4 "C"))
; Построение семейства квадратов:
(DEFUN QN ( ) ; Заголовок функции
(SETQ B 0.0) ; Задан начальный угол B=0
(ID) ; Выполнена функция ID
(WHILE (<= B (* 2 PI)); Начало цикла
; Выполним функцию QUADR с
; параметрами L, B
(QUADR L B)
; Увеличим угол на 0.1*PI
(SETQ B (+B (* PI 0.1)))
; Изменим L в 0.9 раз
(SETQ L (* L 0.9))
); Конец цикла
)
```

Результат выполнения программы 5 показан на рис.1. В предыдущих примерах каждая программа содержала по одному описанию функции. Пример программы 5 демонстрирует, что такое соответствие не обязательно. Программа может содержать несколько описаний функций (несколько функций *DEFUN*).

Каждое описание функции – это функциональный модуль, который можно использовать отдельно, в частности, в других программах. Обратим внимание на передачу данных при вызове функции. В заголовке функции *QUADR* описано два аргумента: *L*, *A*. При обращении в функцию *QUADR* передаются значения параметров: *L* и *B*. Это следует понимать так, что в момент обращения к *QUADR* *A* становится равно *B*; *L* (которое будет работать внутри *QUADR*) становится равным *L* (которое существовало вне *QUADR*).

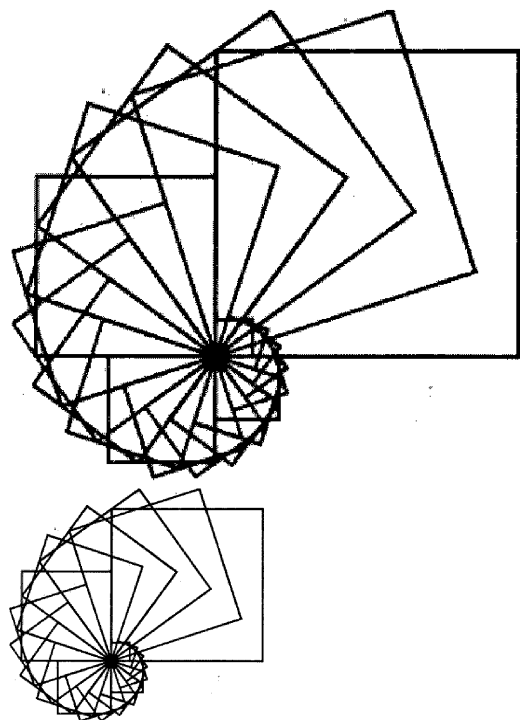


Рис. 1. Результат двукратного выполнения программы 5 с разными исходными данными.

Следовательно, *A* и *B*, *L* и *L* – это разные данные, и не имеет значения, выбраны для их обозначения разные имена (как *A* и *B*) или одинаковые (как *L*). Изменения, которые происходят с *B* и *L* внутри функции, никак не отражаются на значениях *A* и *L* в остальной программе.

Программа 5 иллюстрирует общую схему построения программ для получения параметризованных чертежей:

- ввод исходных данных (в том числе базовой точки);

- определение точек внутри программы с помощью функции *POLAR* с использованием введенных данных;

- построение примитивов на экране с помощью функции *COMMAND* по определенным точкам.

Рассмотрим еще одну функцию для организации цикла в *LISP*-программе. Функция *REPEAT* выполняет операцию цикла с фиксированным количеством повторений:

(REPEAT <количество> [<выражение1>...]) .

Типы аргументов: *<количество>* – целое число, *<выражение1>* – любое выражение. После аргумента *<выражение1>* могут идти другие выражения, которые нужно выполнить внутри цикла. Возвращаемое значение – значение последнего вычисленного выражения. Если *<количество>* имеет значение 0, или отрицательно или после аргумента *<количество>* не заданы выражения, то функция *REPEAT* возвращает значение *NIL*.

Воспользуемся функцией *REPEAT* для вычисления факториала. Программа в данном случае может выглядеть следующим образом:

Пример:

; Вычисление факториала с помощью функции REPEAT

(SETQ N 10); Задайте N

(SETQ I 1 FACTORIAL 1)

(REPEAT (1- N)

(SETQ I (1+ I))

(SETQ FACTORIAL (FACTORIAL I))*

); конец REPEAT.

Поскольку входные значения *I=1* и *FACTORIAL=1!=1*, то остается умножить *FACTORIAL* на *2,3,...N*. Количество таких умножений равно *N-1*, что на языке *Visual LISP* записывается как *(1- N)*. Остальное работает как в предыдущем примере вычисления факториала с помощью функции *WHILE*.

2. ЭТАПЫ ВЫПОЛНЕНИЯ ЛАБОРАТОРНОЙ РАБОТЫ

2.1. Изучите функции ветвления и организации циклов.

2.2. Изучите принцип построения программ с ветвлением и циклами.

2.3. Изучите программу 5.

2.4. Напишите ЛИСП-программу вычерчивания семейства геометрических фигур в соответствии с индивидуальным вариантом.

2.5. В среде *NanoCAD* проведите отладку своей программы.

3. ПЛАН ОТЧЕТА

1. Титульный лист.

2. Цель работы.

3. Задание.

4. Описание программы.

ВАРИАНТЫ ИНДИВИДУАЛЬНЫХ ЗАДАНИЙ ДЛЯ ЛАБОРАТОРНОЙ РАБОТЫ № 10

№ п/п	Параметры		
	Число фигур	Координаты точки P_I	Угол поворота
1	10 правильных треугольников	120, 150	$0,2*\pi$
2	15 квадратов	120, 120	$-0,2*\pi$
3	4 правильных шестиугольника	150, 150	$0,1*\pi$
4	4 ромба (не квадрата)	100, 100	$-0,1*\pi$
5	5 равнобедренных треугольников	200, 200	$0,3*\pi$
6	5 прямоугольных треугольников (неравнобедренных)	250, 250	$-0,3*\pi$
7	5 равнобедренных прямоугольных треугольников	220, 250	$0,25*\pi$
8	10 равнобедренных трапеций	120, 120	$-0,25*\pi$
9	8 прямоугольных трапеций	300, 150	$0,4*\pi$
10	5 прямоугольников (не квадратов)	120, 250	$-0,4*\pi$
11	6 параллелограммов (не прямоугольников)	320, 350	$0,15*\pi$
12	5 произвольных треугольников (неправильных и неравнобедренных)	350, 350	$-0,15*\pi$
13	6 произвольных (неправильных) четырехугольников	320, 320	$0,35*\pi$
14	22 правильных треугольника	400, 150	$-0,35*\pi$
15	22 квадрата	420, 150	$0,2*\pi$
16	24 правильных шестиугольника	1100, 120	$0,25*\pi$
17	14 ромбов (не квадратов)	500, 220	$0,02*\pi$
18	18 равнобедренных треугольников	300, 1200	$0,12*\pi$
19	12 прямоугольных треугольников (неравнобедренных)	600, 200	$0,23*\pi$
20	5 равнобедренных прямоугольных треугольников	120, 30	$0,26*\pi$
21	11 равнобедренных трапеций	2, 23	$0,2*\pi$
22	12 прямоугольных трапеций	2200, 2300	$0,14*\pi$
23	11 прямоугольников (не квадратов)	150, 120	$0,1*\pi$
24	16 параллелограммов (не прямоугольников)	1200, 300	$0,21*\pi$
25	12 произвольных треугольников (неправильных и неравнобедренных)	1600, 1200	$0,22*\pi$
26	22 произвольных (неправильных) четырехугольника	4000, 800	$0,15*\pi$
27	6 квадратов	400, 400	$0,23*\pi$
28	15 правильных шестиугольника	1000, 200	$0,22*\pi$
29	23 ромба (не квадрата)	2300, 120	$0,22*\pi$
30	11 равнобедренных треугольников	1600, 110	$0,2*\pi$
31	11 прямоугольных треугольников (неравнобедренных)	600, 1000	$0,2*\pi$
32	8 равнобедренных прямоугольных треугольников	230, 180	$0,04*\pi$
33	23 равнобедренные трапеции	1200, 110	$0,26*\pi$
34	23 прямоугольные трапеции	1400, 200	$0,11*\pi$
35	32 прямоугольника (не квадрата)	3000, 200	$0,08*\pi$